

KOCAELİ ÜNİVERSİTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

Hafıza Kartı Oyunu (Memory Game)

C++ ve SDL2 Kütüphanesi ile Grafik Arayüz Geliştirme Projesi

Hazırlayan: Osmancan Uslu

Öğrenci No: [240229060]

Ders: Programlama Laboratuvarı

Sınıf: 1. Sınıf (İkinci Öğretim / Normal Öğretim)

Mayıs 2026

1. Projenin Amacı ve Özeti

Bu proje, yapısal ve nesne yönelimli programlama mantığını grafiksel bir arayüz üzerinde pekiştirmek amacıyla geliştirilmiştir. Proje kapsamında, klasik bir "Hafıza Kartı Oyunu" (Memory Game) tasarımı C++ programlama dili ve **SDL2 (Simple DirectMedia Layer)** kütüphanesi mimarisi kullanılarak hayata geçirilmiştir.

Projenin temel kazanımları arasında; dinamik bellek yönetimi, matris tabanlı oyun mantığı tasarımı, kullanıcı etkileşimlerinin (fare tıklamaları ve pencere kapatma eylemleri) asenkron olarak dinlenmesi ve kalıcı veri saklama (dosya işlemlerine dayalı yüksek skor tablosu) mantığının uygulanması yer almaktadır.

2. Kullanılan Teknolojiler ve Kütüphaneler

Projenin geliştirilmesinde, endüstri standardı olan araçlar ve alt kütüphaneler tercih edilerek çapraz platform uyumluluğu ve yüksek performans hedeflenmiştir:

- **Dil:** C++ (C uyumlu alt yapı ve işaretçi mantığı ağırlıklı kaynak kod)
- **Geliştirme Ortamı (IDE):** Microsoft Visual Studio 2022
- **Grafik API / Kütüphane:** SDL2 (Simple DirectMedia Layer 2.0)
- **Yardımcı Sistemler:** Kalıcı veri depolama için standart dosya akışları (I/O)

3. Proje Mimarisi ve Algoritma

3.1. Veri Yapıları Tasarımı

Oyundaki her bir kartın durumunu, eşleşme bilgisini ve sahip olduğu görsel kimliği (ID) takip edebilmek adına özelleştirilmiş bir yapı grubu (struct) tasarımı kullanılmıştır. Matris tabanlı arayüz, bu yapı dizileri üzerinden dinamik veya statik olarak bellekte konumlandırılmaktadır.

```
struct Kart {  
    int id;           // Kartın eşleşme çiftini belirleyen numara  
    bool acikMi;      // Kartın o an ekranda görünür olma durumu  
    bool eslestiMi;   // Kartın doğru tahmin edilip edilmediği  
    int x, y;         // Kartın ekrandaki koordinatları  
};
```

3.2. Ana Oyun Döngüsü (Game Loop)

Oyun mimarisi, SDL2 kütüphanesinin çekirdeğini oluşturan ve saniyede minimum 60 kez çalışan kararlı bir oyun döngüsüne (Game Loop) dayanmaktadır. Bu döngü üç temel adımdan oluşur:

Döngü Adımı	Açıklama ve Gerçekleştirilen İşlemler
Olay Dinleme (Input)	SDL_PollEvent fonksiyonu yardımıyla kullanıcının fare tıklamaları, kart seçimleri ve pencere kapatma talepleri yakalanır.
Mantık Güncelleme	Seçilen iki kartın ID'leri karşılaştırılır. Eğer kartlar eşleşiyorsa eslestimi bayrakları doğrulanır; eşleşmiyorsa belirli bir süre gecikmeyle kartlar geri kapatılır.
Ekrana Çizim (Render)	Arka plan temizlenir (SDL_RenderClear). Kartların durumuna göre kapalı resim veya açık resim dokuları (texture) SDL_RenderCopy ile ekrana basılır.

3.3. Dosya İşlemleri ve Skor Sistemi

Oyuncunun performansını (hamle sayısı veya harcanan süre) kalıcı olarak saklamak adına local disk üzerinde `highscore.txt` dosyası yönetilmektedir. Oyun başarıyla tamamlandığında mevcut skor, dosyada saklanan en iyi skorla mukayese edilir ve üstünlük durumunda dosya yeni verilerle güncellenir.

4. Testler ve Ekran Görüntüleri

Uygulamanın farklı senaryolarda doğru çalışıp çalışmadığı, bellek sızıntısı (memory leak) kontrolleri ve eşleşme mantığı test edilmiştir. Aşağıdaki alanlara uygulamanıza ait arayüz fotoğraflarını ekleyebilirsiniz:

[Ekran Görüntüsü 1: Oyun Başlangıç Ekranı - Tüm Kartların Kapalı Durumu]

[Ekran Görüntüsü 2: Oyun Esnası - İki Kartın Seçilmesi ve Eşleşme Kontrolü]

5. Sonuç ve GitHub Bağlantısı

Bu çalışma sonucunda, alt seviye grafik kütüphanelerinin çalışma mantığı kavranmış ve donanım hızlandırmalı render sistemleri üzerine pratik bir deneyim kazanılmıştır. Projenin kaynak kodları, versiyon kontrol sistemi (Git) standartlarına uygun şekilde temizlenerek uzaktaki depoya aktarılmıştır.

Projenin tüm kaynak kodlarına, kütüphane bağımlılıklarına ve güncel versiyonlarına aşağıdaki adresten erişilebilmektedir:

GitHub Repository: <https://github.com/can49095775148cu-art/MemoryGame/tree/main/MemoryGame>